

PROPUESTA DE PROYECTO

De PDF a Radicado: RPA para Incapacidades Laborales

Motor de automatización OCR + ADRES + RETHUS + RPA EPS integrado a la plataforma Laravel existente

Sponsor / Producto	Incapacidades.ai
Plataforma existente	Laravel: fuente de verdad, persistencia, estados, archivos, credenciales, eventos y auditoría
Nuevo componente	Microservicio FastAPI o Node.js para automatización externa
Resultado esperado	Procesar y radicar incapacidades laborales con trazabilidad, evidencia y actualización en Laravel/Horizon

1. Contexto y oportunidad

Incapacidades.ai ya cuenta con una plataforma Laravel para gestionar empresas, trabajadores, incapacidades, documentos, estados, credenciales, entidades, médicos, eventos y trazabilidad. El reto no busca crear una nueva plataforma, sino construir un componente externo de automatización que se integre con lo existente.

La oportunidad es automatizar el flujo operativo que hoy requiere revisión manual: leer documentos médicos, extraer datos, validar EPS en ADRES según la fecha de la incapacidad, validar médicos en RETHUS y radicar ante portales oficiales de EPS. El retorno esperado es reducir digitación, errores y tiempos de radicación, aumentando trazabilidad y recuperación de cartera.

Principio central: Laravel gobierna y persiste. El microservicio automatiza y responde. Python/Node no debe escribir directamente en la base de datos ni crear una plataforma paralela.

2. Objetivo del proyecto

Construir un microservicio de automatización, en FastAPI o Node.js, que reciba jobs por HTTP POST desde Laravel, ejecute el procesamiento externo y reporte progreso/resultados hacia Laravel para seguimiento desde Horizon.

- Laravel guarda documento
- Laravel crea caso preliminar
- Laravel envía job a microservicio
- Microservicio ejecuta OCR
- Microservicio valida ADRES
- Microservicio valida RETHUS
- Laravel recibe resultados y permite validación humana
- Laravel envía job operativo:
 - A) solicitud_transcripcion
 - B) eps_radicacion
- Microservicio ejecuta RPA en portal EPS
- Microservicio captura evidencia
- Microservicio responde a Laravel
- Laravel actualiza estado, logs, radicado/solicitud y evidencias

3. Alcance del reto

IN SCOPE	OUT OF SCOPE
• Microservicio FastAPI o Node.js. • Recepción de jobs por POST desde Laravel. • Procesamiento concurrente de múltiples incapacidades. • OCR sobre PDF/JPG/PNG. • Validación ADRES usando documento del trabajador y fecha_inicio. • Validación RETHUS del médico. • Adaptadores RPA para mínimo 3 EPS oficiales. • Validación de tamaño máximo por EPS. • Reporte de progreso/resultados a Laravel/Horizon. • Evidencias, logs, errores y trazabilidad.	• Crear una nueva plataforma web. • Crear autenticación, usuarios o multiempresa desde cero. • Crear dashboard financiero. • Liquidar económicamente incapacidades. • Conciliar pagos. • Automatizar todas las EPS del país. • Integración directa con nómina.

4. Responsabilidades por componente

Laravel existente	Microservicio FastAPI / Node.js
Fuente de verdad, persistencia, estados, usuarios, empresas, trabajadores, incapacidades, archivos, credenciales, auditoría, Horizon, validación humana y callbacks.	OCR, validación ADRES, validación RETHUS, RPA en portales EPS, adaptadores, captura de evidencias, manejo de errores y respuesta estructurada.

5. Flujo operativo propuesto

1. Usuario carga PDF/JPG/PNG en Laravel.
2. Laravel guarda archivo y crea caso preliminar en estado pendiente_extraccion.
3. Laravel despacha un job Horizon y envía POST al microservicio.
4. Microservicio responde 202 Accepted y procesa en segundo plano.
5. Microservicio descarga adjuntos desde URL temporal.
6. Microservicio ejecuta OCR y extrae datos de trabajador, incapacidad, EPS, diagnóstico y médico.
7. Microservicio valida EPS en ADRES usando tipo_documento, numero_documento y fecha_inicio extraída.
8. Microservicio valida médico en RETHUS.
9. Microservicio reporta progreso y resultado a Laravel por callback_url.
10. Laravel guarda resultados como borrador, actualiza EPS según ADRES, crea/actualiza médico según RETHUS y deja caso pendiente de validación humana.
11. Operador valida/corriga en Laravel y aprueba la radicación.
12. Laravel envía job de radicación al microservicio.
sistema define proceso:
 13. └─ solicitud_transcripcion
 14. └─ eps_radicion
15. Microservicio selecciona adaptador EPS, valida tamaño máximo, entra al portal oficial, diligencia formulario y sube adjuntos.
16. Microservicio captura radicado, comprobante o screenshot y responde a Laravel.
17. Laravel actualiza estado, radicado, evidencias, eventos y seguimiento en Horizon.

6. Procesos soportados

6.1 Solicitud de transcripción

Aplica cuando la incapacidad necesita ser transcrita, reconocida o registrada previamente por la EPS.

Resultado esperado:

- Número de solicitud de transcripción, si el portal lo entrega.
- Comprobante, si existe.
- Screenshot final obligatorio.
- Estado sugerido: transcripcion_solicitada.

Flujo:

1. Laravel envía job solicitud_transcripcion
2. Microservicio selecciona adaptador EPS
3. Valida adjuntos requeridos
4. Ingresa al portal EPS
5. Diligencia formulario
6. Carga soportes
7. Captura número de solicitud/evidencia
8. Responde a Laravel

6.2 Radicación de incapacidad

Aplica cuando la incapacidad ya está lista para ser presentada ante la EPS.

Resultado esperado:

- Número de radicado, si el portal lo entrega.
- Comprobante, si existe.
- Screenshot final obligatorio.
- Estado sugerido: radicada.

Flujo:

1. Laravel envía job eps_radicacion
2. Microservicio selecciona adaptador EPS
3. Valida tamaño máximo del archivo
4. Ingresa al portal EPS
5. Diligencia formulario
6. Carga soportes
7. Captura radicado/evidencia
8. Responde a Laravel

7. Contrato de integración

El microservicio debe exponer un endpoint único para recibir jobs. Laravel no debe esperar a que el proceso termine en la misma petición; el microservicio debe responder rápido y reportar avances por callback.

POST /automation/jobs
 Authorization: Bearer INTERNAL_TOKEN
 Content-Type: application/json

Respuesta inmediata esperada: HTTP 202 Accepted

7.1 Payload base Laravel → Microservicio

```
{
  "job": {
    "uuid": "job-123",
    "type": "ocr_adres_rethus | eps_radizacion",
    "callback_url": "https://laravel-app.com/api/internal/automation/jobs/job-123/result"
  },
  "empresa": {
    "id": 33,
    "tipo_identificacion": "NIT",
    "numero_identificacion": "900123456",
    "razon_social": "EMPRESA SAS",
    "correo_contacto": "nomina@empresa.com",
    "telefono_contacto": "3000000000"
  },
  "trabajador": {
    "id": 789,
    "tipo_documento": "CC",
    "numero_documento": "123456789",
    "nombres": "JUAN CARLOS",
    "apellidos": "PEREZ LOPEZ",
    "eps_actual": "SANITAS",
    "tipo_cotizante": "DEPENDIENTE",
    "ibc": 2500000
  },
  "caso_preliminar": {
    "id": 456,
    "uuid": "inc-456",
    "estado_actual": "pendiente_extraccion"
  },
  "adjuntos": [
    {
      "tipo": "CERTIFICADO_INCAPACIDAD",
      "archivo_id": 1001,
      "nombre_original": "incapacidad.pdf",
      "mime_type": "application/pdf",
      "size_kb": 850,
      "url_temporal": "https://s3.amazonaws.com/archivo-firmado.pdf"
    }
  ]
}
```

7.2 Callback de progreso Microservicio → Laravel

```
{
  "job_uuid": "job-123",
  "incapacidad_uuid": "inc-456",
  "event": "ocr_started",
  "status": "running",
  "progress": 20,
  "message": "OCR iniciado"
}
```

7.3 Callback final exitoso

```
{
  "job_uuid": "job-123",
```

```

"incapacidad_uuid": "inc-456",
"event": "job_completed",
"status": "success",
"result": {
  "ocr": { "status": "processed" },
  "adres": { "status": "success", "eps_encontrada": "SANITAS" },
  "rethus": { "status": "success", "rethus": "VALIDADO" },
  "radicacion": { "numero_radicado": "RAD-123456" }
}
}

```

8. Reglas de negocio clave

RN-01 — Laravel es la fuente de verdad: Laravel crea casos, envía jobs, recibe callbacks y actualiza datos. El microservicio no escribe directo en la base de datos.

RN-02 — Caso preliminar antes del OCR: Al cargar el documento, Laravel crea un registro mínimo en pendiente_extraccion. La incapacidad se completa luego de OCR, ADRES, RETHUS y validación humana.

RN-03 — OCR no es definitivo: Los datos extraídos se guardan como borrador. Si hay baja confianza o inconsistencias, el operador valida/corriges.

RN-04 — ADRES define EPS operativa: ADRES debe consultarse con tipo_documento, numero_documento y fecha_inicio de la incapacidad. Si devuelve EPS válida, Laravel actualiza la EPS de la incapacidad y registra advertencias si no coincide.

RN-05 — RETHUS actualiza médicos: Según RETHUS, Laravel crea/actualiza el médico, guarda estado y lo asocia a la incapacidad.

RN-06 — Concurrencia obligatoria: El microservicio debe procesar más de una incapacidad al tiempo, aislando logs, archivos temporales, sesiones y resultados por job_uuid.

RN-07 — Reporte hacia Laravel/Horizon: El microservicio debe reportar eventos de progreso a Laravel; Laravel refleja el estado en sus jobs y colas administradas por Horizon.

RN-08 — Adaptadores EPS: Cada EPS debe tener un adaptador independiente. El mínimo del reto son 3 adaptadores funcionales.

9. Eventos mínimos de progreso

```

job_received
ocr_started
ocr_finished
adres_started
adres_finished
rethus_started
rethus_finished
rpa_started
rpa_finished
job_failed
job_completed

```

10. Reglas adaptadores, EPS evaluables y límites de archivo

Regla principal

Mínimo 3 adaptadores EPS funcionales.

Cada adaptador debe soportar al menos uno de estos flujos:

- solicitud_transcripcion
- eps_radicacion

No cuentan como adaptadores funcionales:

- Adaptadores vacíos.
- Adaptadores mock.
- Adaptadores sin interacción con portal oficial.
- Adaptadores que no capturen evidencia.

EPS	Tamaño máximo
Compensar	900 KB
EPS Sura	4 MB
Coosalud	1.1 MB
Sanitas	12 MB
Nueva EPS	10 MB
Salud Total	4 MB
Famisanar	5 MB
Mutual Ser	1 MB

11. Procesamiento concurrente

El microservicio debe poder procesar **más de una incapacidad al tiempo**.

Requisitos mínimos:

- No bloquear el servicio por un solo job.
- Permitir ejecución concurrente controlada.
- Reportar estado individual por job.
- Evitar mezclar evidencias entre jobs.
- Mantener trazabilidad por job_uuid e incapacidad_uuid.

12. Seguridad y operación

- Comunicación Laravel ↔ microservicio con token interno o firma HMAC.
- El microservicio no debe conectarse directamente a MySQL.
- Usar credencial_ref en vez de contraseña plana cuando sea posible.
- No imprimir credenciales, tokens ni URLs sensibles completas en logs.
- URLs de adjuntos temporales y evidencias en storage seguro.
- Cada callback debe incluir job_uuid e incapacidad_uuid.
- Validar callback_url mediante allowlist o firma.
- Aislar sesiones de navegador por job/adaptador para evitar cruces entre empresas o incapacidades.

13. Entregables esperados

- Microservicio FastAPI o Node.js ejecutable.
- Endpoint POST /automation/jobs.
- Procesamiento concurrente de múltiples incapacidades.
- OCR funcional sobre PDF/JPG/PNG.
- Validación ADRES usando fecha_inicio.
- Validación RETHUS.
- Mínimo 3 adaptadores EPS funcionales.
- Validación de tamaño por EPS.
- Callbacks de progreso y resultado hacia Laravel/Horizon.
- Captura de evidencias por intento.
- README técnico y guía para agregar nuevas EPS.

14. Definition of Done

- Laravel puede enviar un job por POST al microservicio.
- El microservicio responde 202 Accepted y procesa en segundo plano.
- El microservicio puede procesar más de una incapacidad al mismo tiempo.
- El microservicio descarga adjuntos mediante URL temporal.
- El microservicio ejecuta OCR y devuelve datos estructurados.
- El microservicio valida EPS en ADRES usando fecha_inicio.
- El microservicio valida médico en RETHUS.
- Laravel puede guardar resultados OCR, actualizar EPS y crear/actualizar médico.
- Laravel envía job de radicación luego de validación humana.
- El microservicio selecciona adaptador EPS y valida tamaño de archivo.
- El microservicio ejecuta RPA en mínimo 3 portales EPS.
- El microservicio captura radicado, comprobante o evidencia.
- El microservicio reporta progreso y resultado final a Laravel.
- Laravel puede actualizar estado, radicado, logs, evidencias y seguimiento en Horizon.